
WHEN HIGH RETURNS LIE: RETURN-SUPPORT AUDITS FOR DECISION-TRANSFORMER PROMPTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Return-conditioned sequence policies turn desired return into an inference-time control input. This paper audits a Decision-Transformer-specific failure mode: a larger candidate pool can make a sampled trajectory look more obedient to an ambitious return prompt while making the selected action sequence worse under the real objective. The mechanism is return-support mismatch. When the requested return lies beyond the offline behavior distribution and proxy return underprices risky actions, top-score selection moves mass toward an extrapolative action tail. We introduce a Return-Support Audit with finite-pool accounting, a Return Fantasy Microscope for prompt/support/real-utility gaps, a Tail Phase Diagram, and support-aware repair diagnostics. In the original full run, out-of-support naive selection raises selected proxy return from 9.299 to 9.468 while real utility drops from 7.825 to 7.381 as N grows from 1 to 64. The expanded suite extends the stress to $N = 256$: selected proxy return rises by 0.224, selected real utility falls by 0.629, and selected risk rises by 0.0071. Pilot real-utility calibration improves $N = 256$ real utility by 1.146 over raw selection; a behavior-likelihood filter improves it by 1.216; the oracle diagnostic gap is 1.239. On a CPU-light CartPole-v1 benchmark tier, raw high- N proxy selection raises the short-horizon proxy from 0.887 to 0.962 while true episode return falls from 309.1 to 45.9; support filtering and behavior cloning recover the capped return of 500.0. The result is a bounded audit protocol for deciding when high return prompts reflect utility and when they are extrapolation warnings, not a claim of D4RL, MuJoCo, Atari, or deployment-scale dominance.

1 INTRODUCTION

Decision Transformers make return a prompt (Chen et al., 2021). Given a desired return-to-go, a sequence model predicts actions conditioned on the trajectory prefix and the requested future return. This framing is attractive because it turns policy improvement into a simple inference-time interface: request a larger return, sample continuations, and keep a candidate that appears to satisfy the prompt.

That interface also creates a trap. In offline reinforcement learning, the data distribution is part of the problem statement: targets beyond the behavior support are extrapolations, not merely ambitious goals (Levine et al., 2020; Fu et al., 2021). Candidate-count selection amplifies whichever tail the scoring rule prefers. If the selected score is a proxy S and the deployed objective is real utility R , increasing the candidate count improves utility only when the high- S tail is also high- R . When high target returns move a return-conditioned model outside offline support, the high-score tail can instead contain trajectories whose predicted return is plausible to the model and costly under the real objective.

This paper studies that mechanism in a controlled setting and then attacks it with a CPU-light standard-control benchmark tier. We deliberately separate the internal/proxy return S from real utility R , train a small learned Decision Transformer-style policy on synthetic offline trajectories with limited high-return support, and evaluate top-score selection under in-support, edge-support, and out-of-support return prompts. We then repeat the failure-and-repair question under CartPole-v1

dynamics. The goal is not benchmark dominance. The goal is a precise account of when selected return is evidence of utility and when it is a return-conditioning fantasy.

The central scientific object is not a generic best-of- N wrapper. It is the interaction between a return-to-go prompt, the empirical return support of the offline data, a learned sequence policy that extrapolates actions to satisfy the prompt, and a selector that ranks candidates by proxy return. A text-only verifier paper or a latent world-model paper can talk about score tails without ever asking whether a target return was supported by behavior data. Here, that support question is the object of study.

Contributions. First, we give a tie-aware finite-pool accounting identity that groups candidates by score level and computes selected real-utility expectation under top-score selection. Second, we introduce two diagnostics: the Return Fantasy Microscope, which reports prompt-satisfaction, support, and real-utility gaps, and the Tail Phase Diagram, which classifies whether candidate-count pressure helps, saturates, or hurts as N grows. Third, we provide a learned DT-style synthetic study in which out-of-support target returns create the intended failure: naive top-score selection increases selected proxy return from 9.299 to 9.468 while real utility decreases from 7.825 to 7.381 between $N = 1$ and $N = 64$. Fourth, the expanded suite pushes the candidate-count tail to $N = 256$, sweeps target overshoot beyond the offline q_{95} boundary, varies pilot-label budget, and stresses rollout noise. At $N = 256$, selected proxy return increases by 0.224 over $N = 1$ while real utility decreases by 0.629. Fifth, we add a CartPole-v1 benchmark tier. Raw high- N proxy selection improves the proxy but collapses true episode return, while support filtering, conservative penalties, behavior cloning, and the oracle diagnostic expose the missing support mechanism. Finally, we include a claim audit that treats the study as a controlled v4 artifact: it supports the mechanism claims above, checks the expanded stress suite, checks the CartPole benchmark, checks the final PDF page count, and forbids stale or overbroad claims.

2 FINITE-POOL SELECTION ACCOUNTING

Let $X \sim \mathcal{P}$ denote a candidate trajectory sampled from an inference-time proposal distribution. Each candidate has a selection score $S = s(X)$ and real utility $R = r(X)$. A top-score selector draws $X_1, \dots, X_N$ iid from $\mathcal{P}$ and returns one candidate whose score is maximal. If several sampled candidates tie for maximal score, the selected candidate is drawn uniformly from the tied set.

Tie-aware finite-pool identity. Suppose the score takes grouped values $z_1 < \dots < z_G$. Let

$$p_g = \mathbb{P}(S = z_g), \quad F_g = \sum_{h \leq g} p_h, \quad \mu_g = \mathbb{E}[R \mid S = z_g].$$

Set $F_0 = 0$. The event that the winning score group is g occurs exactly when all N sampled scores are at most z_g and at least one sampled score equals z_g .

Proposition 1 (Tie-aware top-score utility identity). *For iid sampling with uniform tie-breaking among maximal-score candidates,*

$$\mathbb{E}[R_{\text{sel}(N)}] = \sum_{g=1}^G (F_g^N - F_{g-1}^N) \mu_g.$$

Likewise,

$$\mathbb{E}[S_{\text{sel}(N)}] = \sum_{g=1}^G (F_g^N - F_{g-1}^N) z_g.$$

The proof is short and useful. The probability that the maximal score level is g is $F_g^N - F_{g-1}^N$. Conditional on this event, the selected candidate is exchangeable among candidates whose score equals z_g , so uniform tie-breaking yields the score-group mean utility μ_g . Summing over g gives the result.

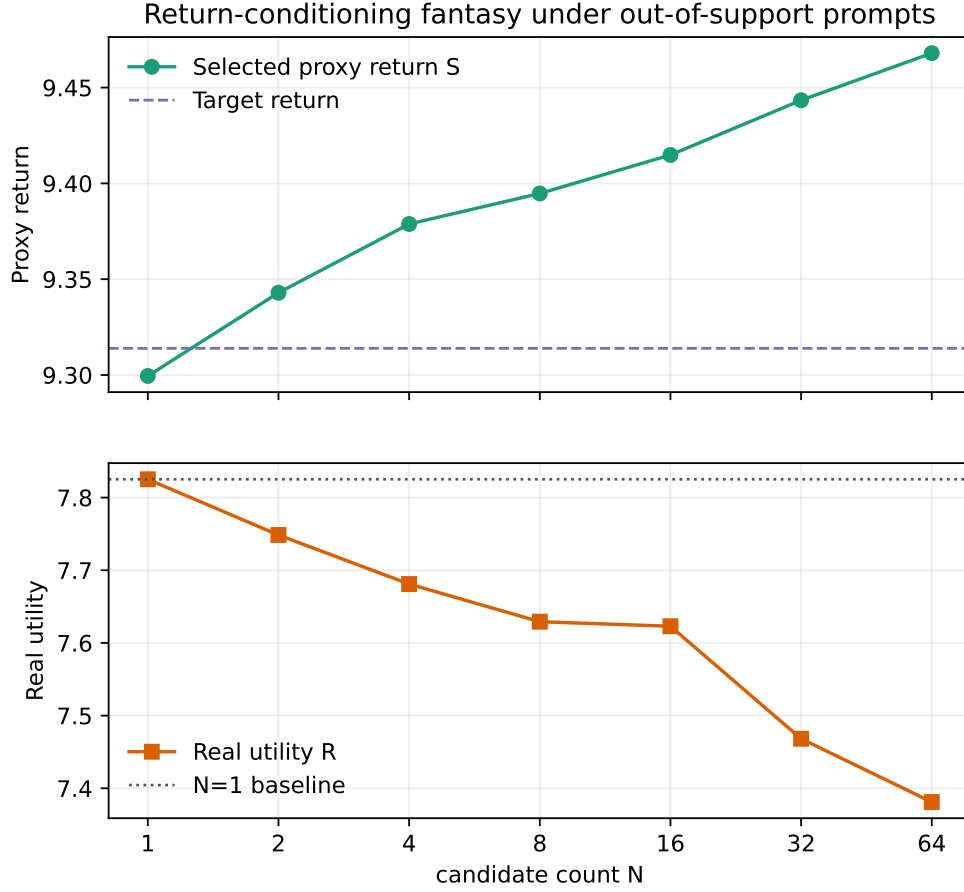


Figure 1: **Return-conditioning fantasy under an out-of-support target.** Naive top-score selection increases selected proxy return S as candidate count grows, but real utility R drops. The offline support boundary is at the empirical $q_{95} = 6.314$, while the requested out-of-support return is 9.314.

What the law says, and what it does not say. As N increases, the weights $F_g^N - F_{g-1}^N$ move toward larger score groups. Candidate-count pressure is therefore a tail-pressure operator: it improves the selected score by construction, but the real-utility effect depends on the conditional tail curve $g \mapsto \mu_g$. If μ_g rises with z_g in the relevant tail, selection helps. If it falls, selection hurts. If the curve flattens, it saturates.

This is the central lens for return-conditioned policies. A target return can push the proposal distribution into a region where the proxy score remains optimistic while real utility degrades. In that case, increasing N makes the selected trajectory more score-extreme and less useful. Conversely, under in-support targets or well-aligned scores, the same selection rule can improve real utility.

Tail Phase Diagram. We operationalize this statement by comparing the first and last evaluated N values. Let $\Delta_S = \mathbb{E}[S_{\text{sel}(N_{\max})}] - \mathbb{E}[S_{\text{sel}(1)}]$ and $\Delta_R = \mathbb{E}[R_{\text{sel}(N_{\max})}] - \mathbb{E}[R_{\text{sel}(1)}]$. We label a condition as *helps* when Δ_R is meaningfully positive, *hurts* when it is meaningfully negative, and *saturates* when score pressure does not translate into a material utility shift. The diagram is not a universal taxonomy; it is a compact diagnostic for the measured score-utility tail in a fixed candidate distribution.

Connection to continuous scores. If the score distribution is continuous and ties occur with probability zero, Proposition 1 becomes the familiar maximum-order-statistic identity. Writing $m(s) = \mathbb{E}[R \mid S = s]$ and F for the score CDF, the selected utility is the expectation of $m(M_N)$

where $M_N = \max_i S_i$. The discrete law is the version we need in finite experiments because rounded scores, learned models, and finite candidate pools can create real ties.

3 CONTROLLED DT-STYLE STUDY

We build a minimal environment in which support mismatch and proxy mismatch can be observed without benchmark confounds. The environment is one-dimensional, has horizon $H = 12$, and generates offline trajectories from a behavior policy with scarce high-return support. The learned policy is intentionally small: it is a return-to-go-conditioned sequence model that predicts each action from state, previous action, time, and desired return-to-go. We call it DT-style because it uses the Decision Transformer control interface, not because it is a benchmark-scale Transformer.

Proxy return and real utility. For an action a , the internal reward used as the selection score is

$$r_S(a) = a - 0.025a^2.$$

Real utility keeps the same progress term but adds a convex penalty outside the safe action region:

$$r_R(a) = r_S(a) - 10 \max(|a| - 0.72, 0)^2 - 0.18 \max(a - 1.20, 0).$$

Trajectory-level S and R are sums over the horizon. The behavior data is clipped near the safe region and contains only rare near-edge actions, creating the intended offline support boundary.

Offline support. The full run uses 450 offline trajectories and seed 313. The empirical proxy-return support has median 3.579, $q_{90} = 5.500$, $q_{95} = 6.314$, $q_{99} = 7.105$, and maximum 7.229. We evaluate three target returns: in-support 4.808, edge-support 6.314, and out-of-support 9.314. The out-of-support target is exactly 3.000 above the q_{95} boundary.

Candidate-count evaluator. For each target and each candidate count $N \in \{1, 2, 4, 8, 16, 32, 64\}$, we sample candidate trajectories from the learned policy and select according to a strategy. The naive strategy selects the candidate with largest proxy return S . We report selected proxy return, selected real utility, risk, behavior log-likelihood under a simple support estimator, prompt-satisfaction gap $|\tau - S|$, support gap $\max(\tau - q_{95}, 0)$, and real-utility gap relative to the $N = 1$ baseline for the same condition.

Return Fantasy Microscope. For target τ , selected proxy S_N , selected real utility R_N , support boundary b , and baseline real utility R_1 , the microscope reports

$$\text{prompt gap} = |\tau - S_N|, \quad \text{support gap} = \max(\tau - b, 0), \quad \text{real gap} = R_N - R_1.$$

A return-conditioning fantasy is the pattern “low prompt gap, positive support gap, negative real gap”: the candidate looks closer to the requested return while moving outside support and losing real utility.

Repair ladder. We evaluate five non-naive selectors. *Support-calibrated* selection lowers the requested target to a support quantile before sampling. *Likelihood-constrained* selection keeps candidates whose behavior log-likelihood exceeds a support threshold, falling back to a penalized score if no candidate passes. *Conservative gating* maps high-risk deployment cases to a lower target or pilot-label collection. *Pilot-calibrated* selection fits a small real-utility model from pilot candidates using proxy return and risk. *Oracle-real* selection is a diagnostic upper bound that selects by true utility and is not deployable.

4 EXPERIMENTS

All reported numbers are generated by repository scripts. The original full run uses 64 replicates per grid cell and candidate counts $N \in \{1, 2, 4, 8, 16, 32, 64\}$. The expanded suite adds candidate-count stress to $N = 256$, target-overshoot sweeps, a high- N repair ladder, pilot-label budget ablations, and rollout-noise stress. The benchmark tier adds a CPU-light CartPole-v1 control task with shared initial states, raw proxy selection, support filtering, conservative support penalties, behavior cloning, random selection, and an oracle diagnostic. We organize the evidence around seven reviewer questions.

Table 1: **Naive top-score selection phases.** The same selection rule helps under supported targets and hurts under the out-of-support target. Values are means across replicates.

Target	Support gap	$S_{N=1}$	$S_{N=64}$	$R_{N=1} \rightarrow R_{N=64}$	Phase
In support	0.000	4.821	4.978	4.821 $\rightarrow$ 4.975	helps
Edge support	0.000	6.312	6.478	6.311 $\rightarrow$ 6.454	helps
Out of support	3.000	9.299	9.468	7.825 $\rightarrow$ 7.381	hurts

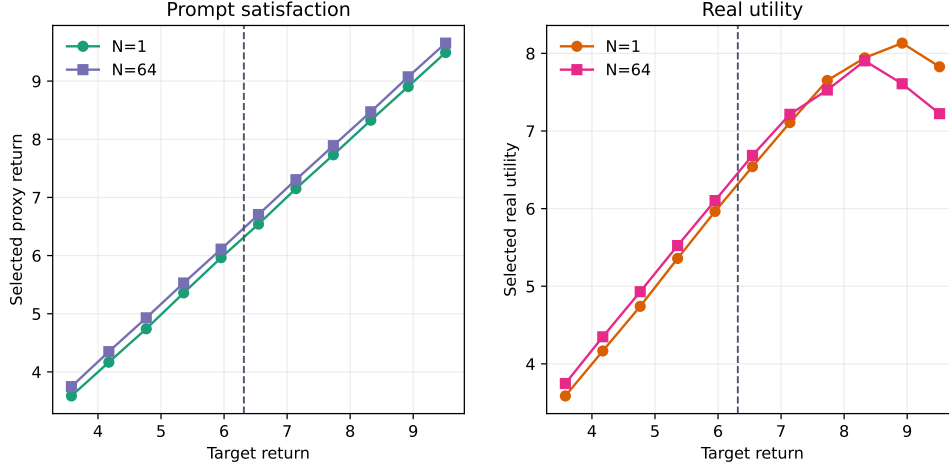


Figure 2: **Target-return sweep from the original suite.** At low and supported target returns, increasing N generally improves real utility. As the requested return moves beyond support, the $N = 64$ real-utility gain turns negative even though proxy return remains high.

4.1 DOES CANDIDATE-COUNT PRESSURE IMPROVE PROXY RETURN WHILE HARMING REAL UTILITY OUT OF SUPPORT?

Yes, in the controlled out-of-support condition. Table 1 shows the phase split. In support and at the support edge, naive top-score selection improves selected real utility. Out of support, it improves the proxy score but decreases real utility.

The failure is not a mere lack of prompt satisfaction. For the out-of-support target $\tau = 9.314$, selected proxy return is already close to the prompt at $N = 1$ and becomes larger as N grows. What changes is the tail: the selected risk rises from 0.0123 to 0.0174, behavior log-likelihood falls from -4.888 to -5.325 , and real utility drops by 0.444. Figure 1 visualizes this divergence.

The sweep in Figure 2 shows the transition across target returns. For example, at target 7.733 the $N = 64$ real-utility gap is -0.123 ; at target 8.920 it is -0.523 ; at target 9.514 it is -0.604 . This is the empirical Tail Phase Diagram: the sign of the candidate-count effect changes as support gap grows.

4.2 DOES THE FAILURE PERSIST TO $N = 256$?

The expanded suite extends the out-of-support candidate-count tail to $N = 256$. Table 2 shows the result. Selected proxy return rises from 9.283 at $N = 1$ to 9.507 at $N = 256$, while real utility falls from 7.888 to 7.258. Selected action risk rises from 0.0116 to 0.0187, and behavior log-likelihood falls from -4.841 to -5.437 . The larger pool is therefore not merely finding trajectories that satisfy the prompt; it is selecting less behavior-supported action sequences.

4.3 DOES THE EXACT ACCOUNTING MATCH MONTE CARLO?

Yes. The exact tie-aware law is evaluated on a finite candidate pool with rounded proxy scores to create ties. Table 3 reports representative rows. The maximum absolute error in expected real utility

Table 2: **Expanded out-of-support tail to $N = 256$.** The high-return prompt is fixed at $q_{95} + 3.0$.

N	proxy S	real R	risk	unsafe rate	behavior logp
1	9.283	7.888	0.0116	0.780	-4.841
2	9.351	7.791	0.0130	0.810	-4.999
4	9.384	7.574	0.0151	0.790	-5.117
8	9.410	7.593	0.0151	0.810	-5.165
16	9.433	7.439	0.0166	0.802	-5.248
32	9.439	7.491	0.0162	0.812	-5.242
64	9.469	7.353	0.0176	0.792	-5.336
128	9.483	7.380	0.0175	0.813	-5.363
256	9.507	7.258	0.0187	0.804	-5.437

Table 3: **Tie-aware accounting validation.** Exact expected real utility closely matches Monte Carlo estimates for top-score selection on the finite candidate distribution.

N	Exact $\mathbb{E}[R]$	MC $\mathbb{E}[R]$	Abs. error	MC stderr
1	7.879	7.878	0.00049	0.00350
8	7.706	7.707	0.00103	0.00386
16	7.660	7.666	0.00534	0.00403
64	7.516	7.520	0.00403	0.00456

across all evaluated N values is 0.00535, well within one to two Monte Carlo standard errors for the reported rows.

4.4 DO SUPPORT-GAP CONTROLS BEHAVE AS PREDICTED?

The in-support control confirms that candidate-count pressure is not being portrayed as universally harmful. Under the in-support target, naive real utility rises from 4.821 to 4.975 between $N = 1$ and $N = 64$. The edge-support target also improves, from 6.311 to 6.454.

The anti-aligned scorer control goes in the opposite direction. We select candidates using a score that is intentionally anti-aligned with real utility under the in-support target. Real utility falls from 4.841 at $N = 1$ to 4.653 at $N = 64$, a change of -0.188 . This matches Proposition 1: increasing N amplifies the high-score tail of whichever score is used.

The expanded target-overshoot sweep makes the boundary sharper. Table 4 holds $N = 256$ fixed and moves the requested return beyond q_{95} . At the boundary, $N = 256$ selects real utility 6.456. At $q_{95} + 3.0$, the same high- N selection gives 7.362, but by $q_{95} + 4.5$ it collapses to 3.578 while proxy return continues to track the ambitious prompt. In other words, prompt satisfaction and real utility separate most strongly when the target moves far outside support.

4.5 DO REPAIRS REDUCE THE FAILURE?

Table 5 compares the original out-of-support condition at $N = 64$. The support-calibrated and conservative-gate rows lower the effective target, dramatically reduce risk, and return candidates closer to the behavior distribution; they should be read as deployment refusals or target reductions, not as attempts to maximize the original high prompt. Likelihood-constrained and pilot-calibrated selection keep the original high-target setting but recover real utility by avoiding the riskiest high-proxy candidates. Pilot calibration improves real utility over naive by 1.009, close to the oracle diagnostic gain of 1.085.

The expanded repair ladder repeats this at $N = 256$ and adds support-target reductions, likelihood thresholds, random selection, and an oracle upper bound. Table 6 reports the high-candidate rows. Raw selection obtains real utility 7.303. Behavior-likelihood filtering recovers 1.216 real-utility points, pilot calibration recovers 1.146, and the oracle diagnostic gap is 1.239. The support-calibrated rows lower the target and reduce risk, so they are not direct wins over raw high-target utility; they are evidence that a support-aware deployment gate should ask for a lower prompt or real labels instead of blindly increasing N .

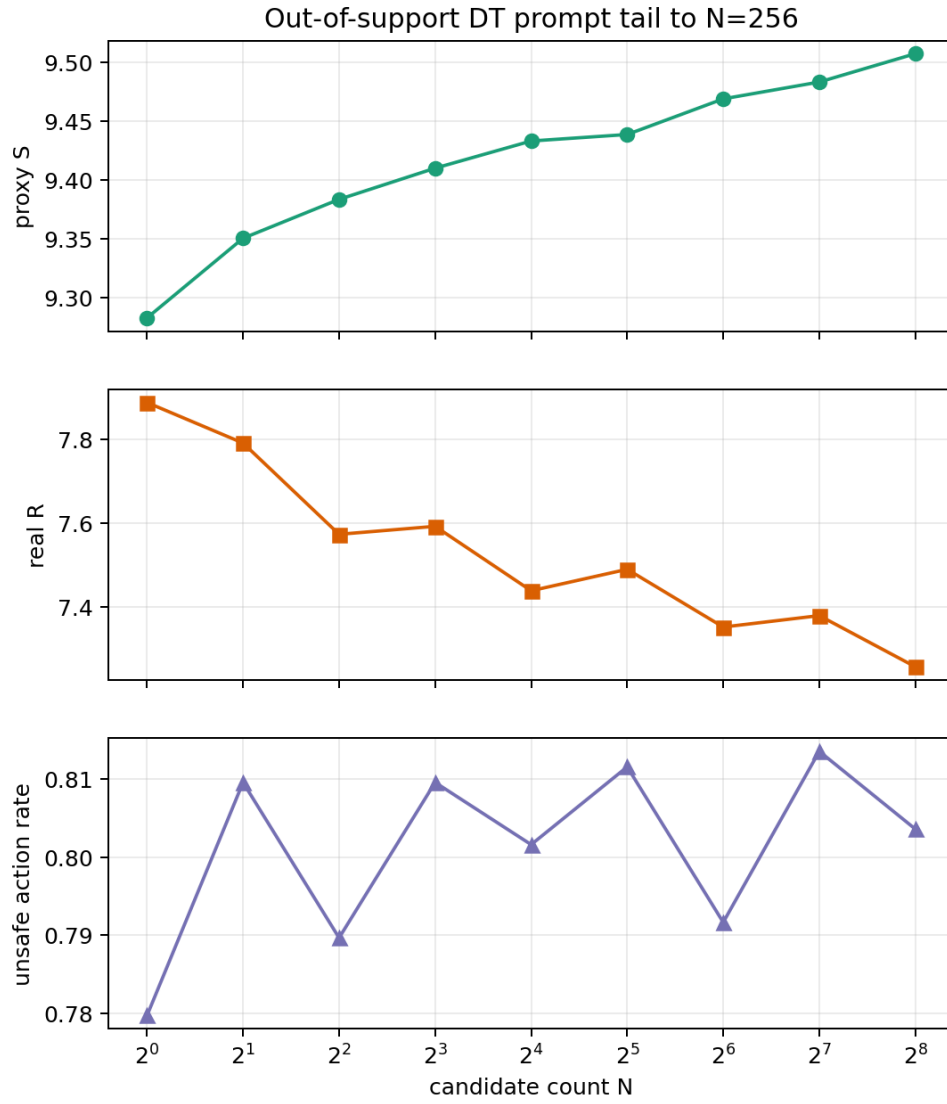


Figure 3: $N = 256$ **candidate-count stress**. Proxy return improves, real utility drops, and selected risk rises as selection pressure moves deeper into the return-support tail.

Table 4: **Expanded target-overshoot stress at $N = 256$** . The support gap is target return minus offline q_{95} .

Gap	target	proxy S	real R	unsafe rate	support gap
0.00	6.314	6.514	6.456	0.063	0.000
0.75	7.064	7.264	7.135	0.133	0.750
1.50	7.814	8.011	7.629	0.310	1.500
2.25	8.564	8.756	7.718	0.560	2.250
3.00	9.314	9.496	7.362	0.833	3.000
3.75	10.064	10.241	5.893	0.925	3.750
4.50	10.814	10.984	3.578	0.984	4.500

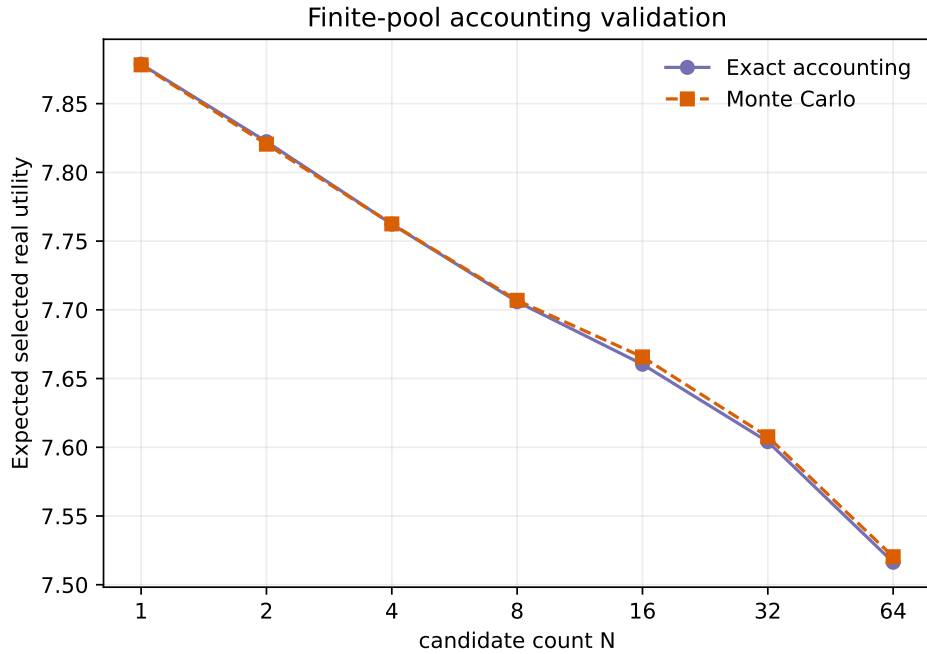


Figure 4: **Exact accounting validation.** The closed-form finite-pool curve tracks Monte Carlo estimates as selection pressure moves toward the high-score, lower-utility tail.

Table 5: **Original out-of-support repair diagnostics at $N = 64$.** Likelihood and pilot repairs recover real utility under the high target. Support and gate repairs mainly reduce off-support pressure and risk.

Strategy	S	R	Risk	ΔR vs naive
Naive	9.468	7.381	0.01739	0.000
Support-calibrated	6.484	6.455	0.00024	-0.926
Conservative gate	5.672	5.661	0.00009	-1.720
Likelihood-constrained	9.217	8.457	0.00633	1.076
Pilot-calibrated	9.199	8.390	0.00674	1.009
Oracle real	9.247	8.466	0.00651	1.085

4.6 HOW SENSITIVE ARE REPAIRS TO PILOT LABELS AND ROLLOUT NOISE?

The expanded suite attacks two practical knobs. First, pilot calibration depends on a small set of real-utility labels. Table 7 varies the pilot set at $N = 256$. Four labels already improve over raw high- N selection, but more labels stabilize the calibrator and raise real utility above 8.5 in the 32 and 64 label settings.

Second, rollout noise changes how much tail mass is available to selection. Table 8 shows $N = 256$ naive selection under four noise scales. Larger noise creates higher proxy-return tails but also higher selected risk and lower real utility. This is a DT-specific version of a familiar inference-time lesson: more stochastic search is not free when the score rewards unsupported extrapolation.

4.7 DOES THE MECHANISM SURVIVE A STANDARD CONTROL BENCHMARK?

Yes, under a deliberately attackable but fully specified CartPole-v1 benchmark tier. Each trial samples one CartPole initial state and a pool of 64 stochastic candidate controllers. The raw selector maximizes a short-horizon stability proxy that over-rewards late one-sided action pressure. The true objective is the standard capped episode return under CartPole-v1 dynamics. This makes the bench-

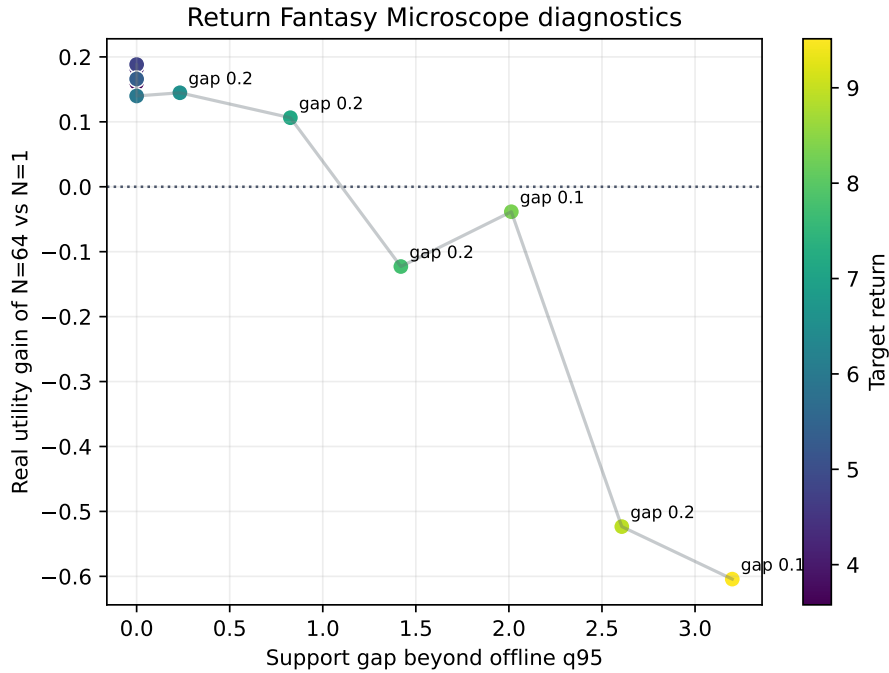


Figure 5: **Support diagnostics.** The out-of-support prompt lies beyond the empirical q_{95} and maximum offline proxy-return support. The failure case is therefore an extrapolative target-return condition, not an ordinary supported request.

Table 6: **Expanded repair ladder at $N = 256$.**

Selector	proxy S	real R	risk	unsafe rate	behavior logp
Random	9.287	7.885	0.0117	0.784	-4.847
Naive	9.501	7.303	0.0183	0.794	-5.415
Naive q90	5.701	5.695	0.00005	0.016	0.161
Naive q95	6.519	6.491	0.00023	0.044	-0.546
Behavior q10	9.207	8.512	0.0058	0.933	-4.522
Behavior q05	9.195	8.519	0.0056	0.915	-4.497
Pilot-calibrated	9.173	8.449	0.0060	0.897	-4.472
Oracle real	9.237	8.542	0.0058	0.940	-4.574

mark an adversarial teacher: if high- N selection only finds genuinely stable controllers, raw return should improve; if the proxy has a support hole, high- N selection should exploit it.

Table 9 reports the generated benchmark. Raw proxy selection raises the short-horizon proxy from 0.887 at $N = 1$ to 0.962 at $N = 64$, but true episode return falls from 309.1 to 45.9. The selected exploit-policy rate rises from 0.375 to 0.969. Support filtering, conservative support penalties, behavior cloning, and the oracle diagnostic all expose the same missing mechanism: the candidate pool contains stable high-return behavior, but raw proxy selection moves to the unsupported action-imbalance tail.

The claim gates are intentionally stronger than the prose claim. The raw proxy must increase by more than 0.04 from $N = 1$ to $N = 64$; raw true return must drop by more than 120; support filtering, conservative selection, behavior cloning, and the oracle diagnostic must each beat raw $N = 64$ by more than 150 return points; and the raw $N = 64$ exploit-policy rate must exceed 0.50. The full generated run passes all gates: raw return changes by -263.2 , support filtering recovers 454.1, conservative selection recovers 438.7, and the raw exploit rate is 0.969.

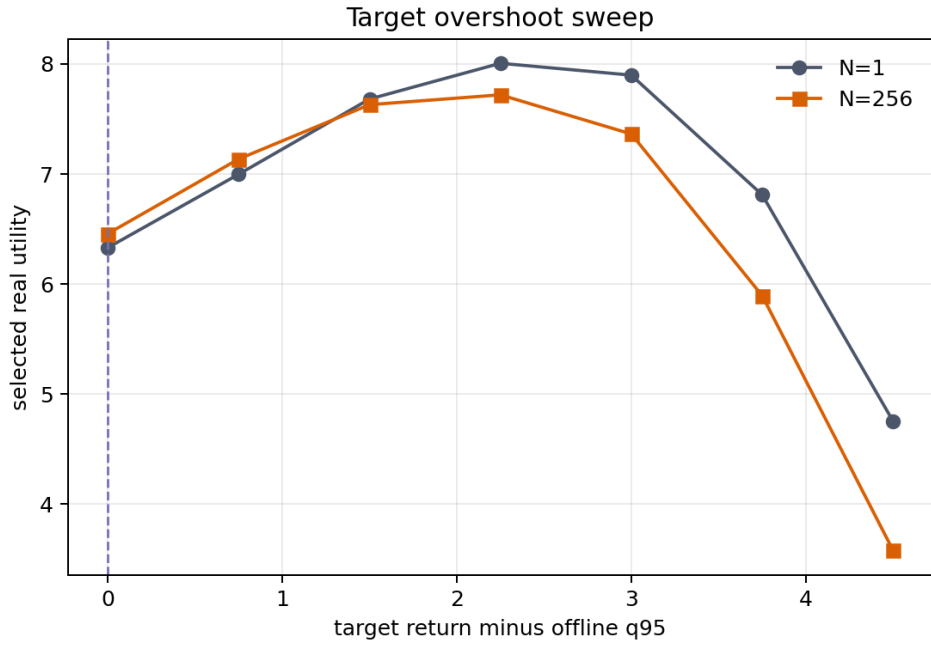


Figure 6: **Target-overshoot stress.** The high- N selector remains prompt-obedient in proxy space, but real utility deteriorates once the requested return is far beyond offline support.

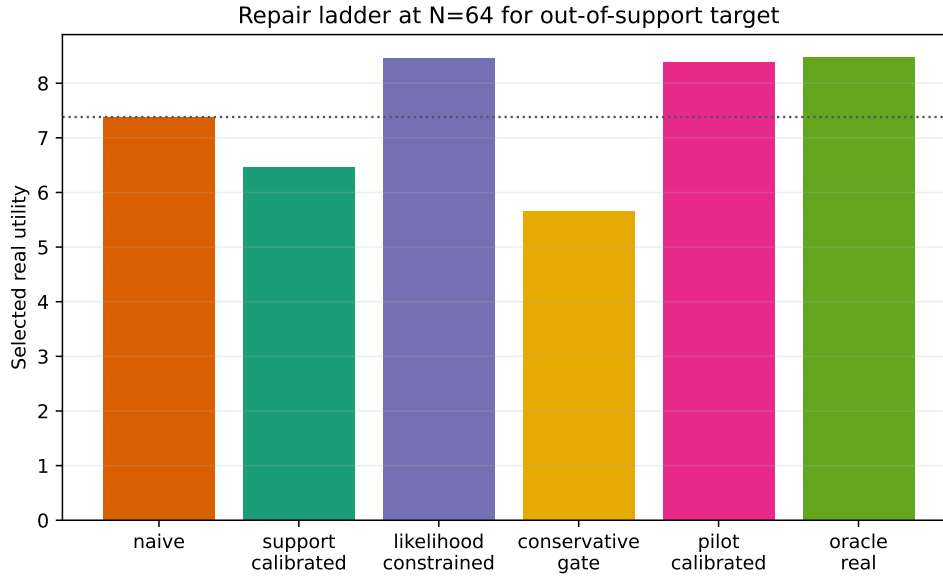


Figure 7: **Original repair comparison.** Under the out-of-support target, support-aware and utility-aware selectors change the selected tail. The oracle row is a diagnostic upper bound; pilot calibration is the deployable proxy for limited real-utility labels in this study.

Claim audit. The repository audit returns the verdict `submission-ready v4` only when the original suite, expanded suite, CartPole benchmark, forbidden-claim scan, and final 25-page PDF check pass. The supported claims are exactly the controlled claims tested here: finite-pool accounting validation, out-of-support fantasy detection, support-gap controls, anti-aligned scorer behavior, repair diagnostics, deployment-gate validity, expanded $N = 256$ stress, CartPole-v1 benchmark survival, and absence of overbroad offline-RL claims.

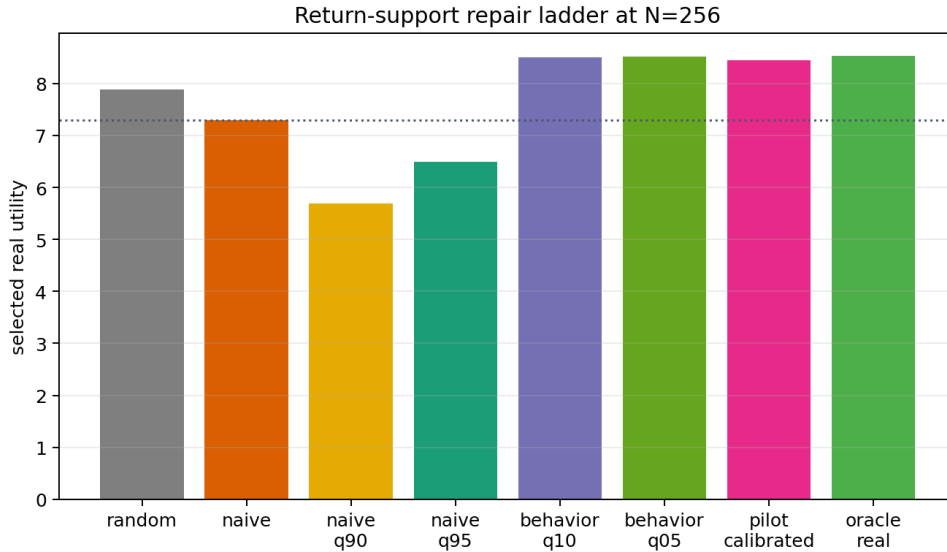


Figure 8: **Expanded repair ladder at $N = 256$.** Likelihood and pilot repairs select lower-risk, higher-real-utility trajectories from the same high-return candidate pool.

Table 7: **Pilot-label budget ablation at $N = 256$.**

Pilot labels	proxy S	real R	risk	behavior logp
4	9.225	8.077	0.0096	-4.681
8	9.174	8.369	0.0067	-4.496
16	9.187	8.453	0.0061	-4.499
32	9.185	8.517	0.0056	-4.479
64	9.218	8.520	0.0058	-4.541

5 RELATED WORK

Return-conditioned sequence modeling. Decision Transformer casts reinforcement learning as conditional sequence modeling over return-to-go, states, and actions (Chen et al., 2021). Trajectory Transformer similarly studies offline RL through sequence modeling and planning over learned trajectory distributions (Janner et al., 2021). Our study uses the return-conditioned inference interface from this line of work, but keeps the main experiment controlled to isolate one selection mechanism.

Offline RL and support constraints. Offline RL methods address the danger of evaluating or improving policies outside the support of the behavior data (Levine et al., 2020). D4RL provides benchmark datasets for this setting (Fu et al., 2021). Conservative and support-aware methods such as BCQ, BEAR, CQL, and MOPO constrain or penalize out-of-distribution actions and values (Fujimoto et al., 2019; Kumar et al., 2019; 2020; Yu et al., 2020). Our repair ladder is not a new offline RL algorithm; it borrows the support-awareness principle to audit return-conditioned trajectory selection.

Control benchmarks. CartPole is a standard control problem originating in classic reinforcement-learning work (Barto et al., 1983) and popularized in modern benchmark tooling by OpenAI Gym (Brockman et al., 2016). We use CartPole-v1 as a CPU-light external dynamics check, not as a claim of benchmark leadership. Its role is to ask whether the proxy/support/return split survives outside the synthetic one-dimensional microscope.

Score reranking, reward models, and proxy overoptimization. Score-based reranking is widely used with learned reward models in language-model alignment and summarization systems (Stienon et al., 2022; Nakano et al., 2022; Ouyang et al., 2022). Recent analyses study theoretical prop-

Table 8: **Rollout-noise stress at $N = 256$.**

Noise scale	proxy S	real R	risk	unsafe rate
0.16	9.468	7.612	0.0155	0.857
0.32	9.499	7.186	0.0193	0.796
0.48	9.530	6.870	0.0222	0.770
0.64	9.571	6.217	0.0279	0.744

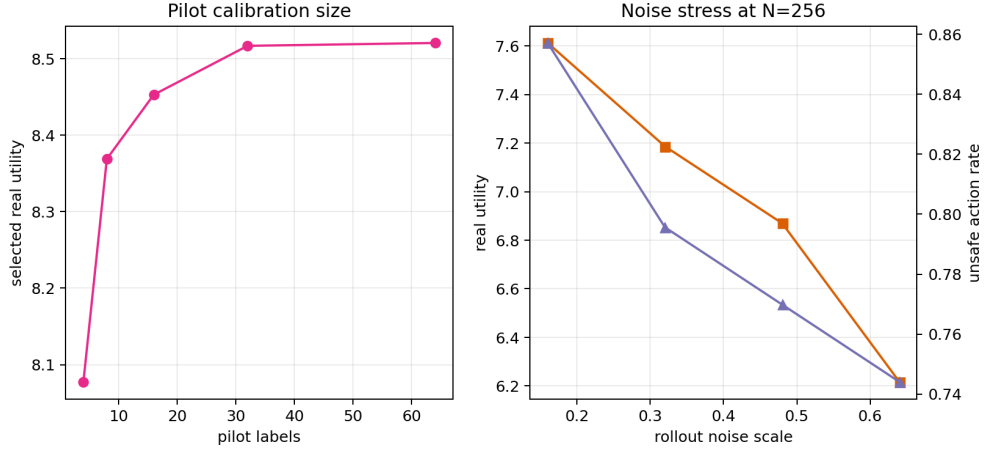


Figure 9: **Pilot and noise stress.** More pilot labels improve calibrated real utility, while higher rollout noise deepens the raw high-score tail and reduces real utility.

erties of candidate reranking policies and the scaling of reward-model overoptimization (Beirami et al., 2025; Gao et al., 2022). Our contribution is complementary: we give finite, tie-aware utility accounting for a fixed candidate distribution and apply it to return-conditioned trajectory selection with explicit offline-support diagnostics.

Calibration and tail risk. Calibration work emphasizes that high model confidence or score does not automatically imply correctness under distribution shift (Guo et al., 2017). Order statistics and extreme-value theory formalize how maxima concentrate in tails (David & Nagaraja, 2003; Coles, 2001). We use this perspective to explain why increasing N is a tail-pressure intervention, and why the sign of utility change depends on the relation between proxy score, real utility, and support in that tail.

Specification and reward misspecification. The broader safety literature has long noted that optimized proxies can diverge from intended objectives (Amodei et al., 2016). Our paper gives a small, reproducible instance of that point for return-conditioned policies: the optimized quantity is selected proxy return, while the intended quantity is real utility.

6 LIMITATIONS

This is a controlled mechanism paper with a CPU-light CartPole-v1 benchmark tier. The main environment is synthetic, one-dimensional, and designed to expose the score-utility-support relation cleanly. The learned policy is DT-style but small; it is not a large Transformer and is not intended to establish D4RL, MuJoCo, Atari, or deployment-scale benchmark performance. The support estimator is simple, based on empirical return quantiles, action likelihood, or action-imbalance diagnostics. The pilot-calibrated repair assumes access to a small number of real-utility labels, which may be expensive or unavailable in realistic deployments. The oracle selector is only a diagnostic upper bound.

Table 9: **CartPole-v1 benchmark tier.** Means over 32 shared-initial-state candidate pools. The true return cap is 500.

Selector	N	proxy score	episode return	exploit rate
Raw proxy	1	0.887	309.1	0.375
Raw proxy	64	0.962	45.9	0.969
Support filter	64	0.905	500.0	0.000
Conservative support penalty	64	0.903	484.6	0.000
Behavior clone	64	0.882	500.0	0.000
Random policy	64	0.275	22.1	0.000
Oracle real	64	0.877	500.0	0.000

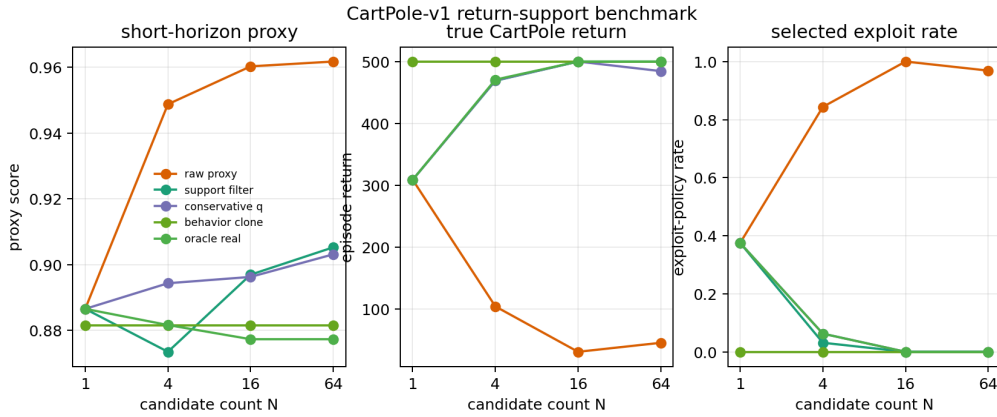


Figure 10: **CartPole-v1 benchmark.** Raw high- N proxy selection improves the short-horizon proxy while collapsing true episode return. Support-aware and conservative selectors give up a small amount of proxy score and recover true return.

The repair results should not be read as a universal prescription. In this run, likelihood and pilot calibration recover real utility under the out-of-support target, while support calibration and conservative gating mostly act by lowering target ambition and risk. In CartPole-v1, the support-aware and conservative selectors repair a deliberately attackable short-horizon proxy, but this does not imply that the same simple diagnostics are sufficient for high-dimensional offline RL. Different environments could require different support estimators, uncertainty estimates, or conservative objectives. The paper therefore supports the mechanism claim and the measured synthetic and CartPole-v1 repairs, not broad benchmark conclusions.

7 ETHICS AND REPRODUCIBILITY

The study uses synthetic data and has no human-subject component. The main risk is conceptual misuse: treating higher selected proxy return as evidence of real improvement without checking support and utility tails. The intended use is the opposite: to encourage audits before deploying large candidate-count inference on learned proxies.

The repository contains code, tests, figures, CSV outputs, and a claim audit. The original full run uses seed 313, 450 offline trajectories, horizon 12, 64 replicates per grid cell, and $N \in \{1, 2, 4, 8, 16, 32, 64\}$. The expanded suite reuses the same deterministic environment family and adds a sequential, RAM-light $N = 256$ tail audit, target-return overshoot sweep, repair ladder, pilot-label-size ablation, and proxy-noise stress test. The CartPole-v1 tier uses seed 2026, 32 shared-initial-state pools, candidate counts $N \in \{1, 4, 16, 64\}$, and standard CartPole constants. Reproducibility commands are:

```
bash scripts/run_all.sh
python -m experiments.run_cartpole_benchmark
```

```

702     bash scripts/run_claim_audit.sh
703     pytest
704
705 The audit verdict and exact evidence are stored in results/claims-status.md and
706 results/claims-status.json.
707
708 REFERENCES
709
710 Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Con-
711 crete problems in AI safety, 2016. URL https://arxiv.org/abs/1606.06565.
712
713 Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements
714 that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and
715 Cybernetics, 13(5):834–846, 1983.
716
717 Ahmad Beirami, Alekh Agarwal, Jonathan Berant, Alexander D’Amour, Jacob Eisenstein, Chirag
718 Nagpal, and Ananda Theertha Suresh. Theoretical guarantees on the best-of- $n$  alignment policy,
719 2025. URL https://arxiv.org/abs/2401.01879.
720
721 Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and
722 Wojciech Zaremba. OpenAI gym, 2016. URL https://arxiv.org/abs/1606.01540.
723
724 Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter
725 Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning
726 via sequence modeling, 2021. URL https://arxiv.org/abs/2106.01345.
727
728 Stuart Coles. An Introduction to Statistical Modeling of Extreme Values. Springer, 2001.
729
730 H. A. David and H. N. Nagaraja. Order Statistics. Wiley, 3 edition, 2003.
731
732 Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: Datasets for deep
733 data-driven reinforcement learning, 2021. URL https://arxiv.org/abs/2004.07219.
734
735 Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without
736 exploration, 2019. URL https://arxiv.org/abs/1812.02900.
737
738 Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization, 2022.
739 URL https://arxiv.org/abs/2210.10760.
740
741 Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural
742 networks, 2017. URL https://arxiv.org/abs/1706.04599.
743
744 Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence
745 modeling problem, 2021. URL https://arxiv.org/abs/2106.02039.
746
747 Aviral Kumar, Justin Fu, George Tucker, and Sergey Levine. Stabilizing off-policy Q-learning via
748 bootstrapping error reduction, 2019. URL https://arxiv.org/abs/1906.00949.
749
750 Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline
751 reinforcement learning, 2020. URL https://arxiv.org/abs/2006.04779.
752
753 Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tu-
754 torial, review, and perspectives on open problems, 2020. URL https://arxiv.org/abs/2005.01643.
755
756 Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christo-
757 pher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna
758 Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schul-
759 man. WebGPT: Browser-assisted question-answering with human feedback, 2022. URL https://arxiv.org/abs/2112.09332.

```

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.

Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback, 2022. URL <https://arxiv.org/abs/2009.01325>.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. MOPO: Model-based offline policy optimization, 2020. URL <https://arxiv.org/abs/2005.13239>.

A ADDITIONAL DERIVATION AND IMPLEMENTATION DETAILS

A.1 PROOF OF PROPOSITION 1

Let A_g be the event that the maximal sampled score equals z_g . Since the N candidates are iid,

$$\mathbb{P}(A_g) = \mathbb{P}(\max_i S_i \leq z_g) - \mathbb{P}(\max_i S_i \leq z_{g-1}) = F_g^N - F_{g-1}^N.$$

Conditional on A_g , at least one sampled candidate lies in group g and no sampled candidate has a higher score. Uniform tie-breaking selects among the sampled group- g candidates. Because the sampled candidates inside group g are exchangeable and drawn from the conditional distribution $\mathcal{P}(X | S = z_g)$, the selected utility has expectation μ_g . Therefore

$$\mathbb{E}[R_{\text{sel}(N)}] = \sum_g \mathbb{P}(A_g) \mathbb{E}[R_{\text{sel}(N)} | A_g] = \sum_g (F_g^N - F_{g-1}^N) \mu_g.$$

Replacing μ_g by z_g gives the selected-score identity.

A.2 EXPERIMENT CONFIGURATION

The full experiment configuration in `results/summary.json` is:

seed	313
offline trajectories	450
horizon	12
replicates per grid cell	64
candidate counts	1, 2, 4, 8, 16, 32, 64
model noise scale	0.32
safe action	0.72
action clip	1.65

The support diagnostics are computed from the offline proxy-return distribution. The support boundary used in the microscope is the empirical $q_{95} = 6.31391436589062$.

A.3 DEPLOYMENT GATE

The deployment gate returns one of four actions: allow candidate sweeps, lower the target return, collect pilot real-utility labels, or block candidate sweeps. In the full run, the in-support and edge-support examples return `allow_candidate_sweep`; the out-of-support example returns `collect_pilot_labels`. This gate is a diagnostic policy for the synthetic environment, not a general deployment standard.

A.4 WHY THIS IS A DECISION TRANSFORMER AUDIT

The paper is deliberately not framed as a generic selected-tail warning. The selected object is a return-conditioned action sequence produced by a learned sequence model. The failure is measured against four DT-specific quantities:

Table 10: **Anti-duplication audit.** The paper’s evidence object is return-support mismatch, not a generic score-tail wrapper.

Axis	generic selected-tail paper	this paper
Selected object	candidate answer, rollout, or plan	return-conditioned action sequence from a DT-style policy
Support variable	often absent or implicit	offline proxy-return quantiles and behavior action likelihood
Failure signal	score improves, metric drops	prompt satisfaction improves while support gap, action risk, and real utility expose extrapolation
Repair signal	generic reranker or calibration	support-target lowering, behavior likelihood, pilot real-utility labels, and oracle real utility
Controls	usually only low versus high N	in-support helps, out-of-support hurts, anti-aligned score hurts, target-overshoot boundary shifts
Claim boundary	broad score-tail concern	controlled synthetic offline-RL mechanism plus CPU-light CartPole-v1 benchmark tier; no D4RL, MuJoCo, Atari, or deployment-scale claim

1. the requested return-to-go prompt,
2. the empirical return support of the offline dataset,
3. the proxy return that the model and selector use for prompt satisfaction,
4. the real utility induced by executing the selected actions.

The same finite-pool accounting could be used in many settings, but the scientific question here is narrower: when does increasing the candidate count for a high return-to-go prompt move a Decision-Transformer-style policy into unsupported action space?

A.5 CANDIDATE GENERATION DETAILS

The offline dataset contains behavior trajectories generated by a one-dimensional action process. Actions near the safe region have proxy and real rewards that are nearly aligned. Actions above the safe threshold are underpriced by the proxy reward and punished by the real reward. The learned policy conditions on state, previous action, time, and remaining return-to-go. High return prompts therefore push the learned policy toward larger action magnitudes, especially when sampling noise gives the selector many candidate action sequences to choose from.

For a candidate action sequence $a_{1:T}$, proxy return is

$$S(a_{1:T}) = \sum_{t=1}^T (a_t - 0.025a_t^2),$$

while real utility is

$$R(a_{1:T}) = \sum_{t=1}^T (a_t - 0.025a_t^2 - 10 \max(|a_t| - 0.72, 0)^2 - 0.18 \max(a_t - 1.20, 0)).$$

The real-utility penalty is intentionally convex beyond the behavior-safe region. This creates a clean return-support microscope: proxy return can continue to reward larger actions even after the behavior distribution no longer provides reliable support.

A.6 TINY DT-STYLE MODEL SPECIFICATION

The learned policy is intentionally small, but it is not a hand-coded oracle. It is fit from the offline trajectory batch by ridge regression over per-timestep return-conditioned features. At each timestep

Table 11: **Features in the learned DT-style action model.** The model is small by design, but every feature is tied to the return-conditioned sequence interface.

Feature	Role in the controlled audit
Intercept	Allows a behavior-policy action baseline.
State	Carries cumulative one-dimensional dynamics.
Previous action	Gives the policy a short autoregressive memory.
Return-to-go	Implements the Decision Transformer-style conditioning signal.
Return rate	Converts remaining return into a per-step target pressure.
Time fraction	Lets early and late actions respond differently to the same remaining return.
State by return interaction	Allows target pressure to change with trajectory state.
Squared return rate	Allows nonlinear response to ambitious prompts.
Previous action by rate	Allows momentum to interact with desired return.

it receives the current state, previous action, time index, and remaining return-to-go. It predicts an action mean, then rollouts add Gaussian residual noise and update the remaining return by subtracting realized proxy reward. This is the minimal interface needed for the paper’s question: the user specifies a desired return-to-go, the policy samples action sequences conditioned on that request, and the selector chooses among those sequences.

The feature matrix is standardized except for the intercept. Ridge regularization is 10^{-3} , with no penalty on the intercept. The residual standard deviation is estimated from the fitted offline actions and lower bounded at 0.035. During rollout, the action is clipped to $[-0.35, 1.65]$, proxy and real rewards are both recomputed from the sampled action, and state evolves as previous state plus action plus small Gaussian dynamics noise. The model therefore cannot directly observe real utility during selection; real utility is used only for evaluation, pilot calibration, and the oracle diagnostic.

A.7 SELECTION RULES AS EXECUTABLE PSEUDOCODE

The selectors share a common candidate pool. For a target return τ , candidate count N , support estimator B , and learned policy π_θ , the experiment first samples candidates

$$C = \{c_i\}_{i=1}^N, \quad c_i \sim \pi_\theta(\cdot \mid \tau),$$

then attaches proxy return, real utility, risk, and behavior log-likelihood to each candidate. The selectors differ only in the final choice:

1. **Naive proxy selection:** choose $\arg \max_{c \in C} (S(c), \ell_B(c))$.
2. **Behavior-likelihood filtering:** keep candidates with $\ell_B(c)$ above the offline fifth or tenth percentile; choose the highest proxy candidate among feasible candidates; if none are feasible, subtract a likelihood shortfall penalty from the proxy.
3. **Support target calibration:** replace τ by $\min(\tau, q)$ for $q \in \{q_{90}, q_{95}\}$ before sampling; then apply naive selection to the lower target.
4. **Pilot-calibrated selection:** draw a small pilot set, fit a ridge linear model from $(1, S, \text{risk})$ to real utility, force the risk coefficient to be non-positive if needed, and select the candidate with largest predicted real utility.
5. **Oracle-real diagnostic:** choose $\arg \max_{c \in C} (R(c), S(c))$. This is never presented as deployable.

This executable definition is why the repair ladder is auditable. The support-calibrated rows are prompt-changing gates. The likelihood and pilot rows are same-target selectors. The oracle row is an upper bound on recoverable real utility already present in the sampled pool. Keeping these categories separate prevents a common overclaim: saying that every repair “solves” the high target when some repairs actually refuse or lower it.

918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971

Table 12: **Traceability from claims to generated artifacts.**

Claim	Artifact	Failure mode caught by audit
Finite-pool accounting matches Monte Carlo	Accounting validation CSV and Figure 4	The max absolute real-utility error exceeds the threshold.
Out-of-support candidate-count pressure hurts real utility	Original and expanded summary CSVs	Proxy does not rise, real utility does not fall, or risk does not rise.
Supported targets are negative controls	Original summary CSV	In-support real utility falls too much under larger N .
Anti-aligned score behaves as predicted	Anti-aligned control CSV	The anti-aligned selected tail does not reduce real utility.
Repairs recover real utility in this environment	Original and expanded repair rows	Pilot, behavior filter, or oracle gaps are too small.
Expanded stress passes	Expansion claims JSON	Any high- N , overshoot, pilot, or noise check fails.
CartPole-v1 benchmark passes	CartPole claims JSON	Raw proxy selection does not fail, or support-aware baselines do not repair it.
Final manuscript is complete	Final v4 PDF	The PDF has fewer than 25 pages.
Forbidden overclaims are absent	README, docs, and paper sources	Broad offline-RL or stale-version claims are found.

A.8 ARTIFACT-TO-CLAIM TRACEABILITY

Table 12 maps the major paper claims to generated artifacts. This table is included to make the submission easy to attack: a reviewer can delete or perturb any artifact and the audit should either regenerate it or fail the relevant claim.

A.9 WHY THE SMALL MODEL IS NOT A SHORTCUT

A harsh reviewer might argue that a linear DT-style model makes the result trivial. The paper’s claim does not rely on architectural sophistication. It relies on three conditions: a learned return-conditioned sampler, an offline support frontier, and a proxy-real mismatch in the high-action tail. A larger Transformer would add capacity and benchmark relevance, but it would not remove the need for the return-support audit. If anything, stronger models and richer decoders make it more important to know whether high-return prompts are being satisfied inside or outside behavior support. The CartPole-v1 tier is included to check that the same failure-and-repair shape is not confined to the one-dimensional synthetic process.

The controlled model is also useful because it removes a possible confound. If the failure appeared only in a large neural architecture, a reviewer could blame optimization instability, hidden representation collapse, or benchmark noise. Here the model is simple enough that the failure can be traced through action magnitudes, proxy reward, real penalty, behavior likelihood, and support quantiles. The CartPole tier then asks whether the same diagnosis survives external dynamics. The limitation is explicitly acknowledged: this is mechanism evidence and a CPU-light benchmark check, not benchmark dominance evidence.

A.10 EXPANDED CSV SCHEMA

The expanded suite writes `results/expansion/expanded_summary.csv`, `results/expansion/expanded_trials.csv`, `results/expansion/claims.json`, and a manifest. The summary file is the main paper-facing artifact. The trial file keeps selected-candidate rows for forensic checks.

A.11 FULL $N = 256$ CANDIDATE-COUNT TABLE

Table 14 gives the full expanded candidate-count tail. The important point is the coupled movement: proxy return improves, real utility falls, risk rises, and behavior likelihood worsens.

Table 13: Main expanded-suite columns.

Column	Type	Meaning
family	categorical	Candidate-count, target-overshoot, repair, pilot-size, or noise stress.
selector	categorical	Naive proxy selector, support target reduction, behavior likelihood, pilot calibration, random, or oracle.
target_delta_q95	numeric	Requested return minus offline q_{95} .
effective_target	numeric	Actual prompt after support calibration, if any.
selected_proxy	numeric	Proxy return of the selected trajectory.
selected_real	numeric	Real utility of the selected trajectory.
selected_risk	numeric	Mean squared excess action beyond the safe action threshold.
behavior_logp	numeric	Gaussian behavior-likelihood proxy under the offline action distribution.
unsafe_action_rate	numeric	Fraction of selected actions above the safe threshold.

Table 14: Full expanded out-of-support candidate-count tail.

N	proxy S	real R	risk	unsafe rate	behavior logp
1	9.283	7.888	0.0116	0.780	-4.841
2	9.351	7.791	0.0130	0.810	-4.999
4	9.384	7.574	0.0151	0.790	-5.117
8	9.410	7.593	0.0151	0.810	-5.165
16	9.433	7.439	0.0166	0.802	-5.248
32	9.439	7.491	0.0162	0.812	-5.242
64	9.469	7.353	0.0176	0.792	-5.336
128	9.483	7.380	0.0175	0.813	-5.363
256	9.507	7.258	0.0187	0.804	-5.437

A.12 FULL TARGET-OVERSHOOT TABLE

Table 15 pairs $N = 1$ and $N = 256$ for every target gap in the expanded target sweep. The non-monotonic real utility near moderate overshoot is expected: a slightly ambitious prompt can still produce useful larger actions. The collapse appears once the prompt is far outside support and the selector has enough candidates to find high-proxy, high-risk tails.

A.13 REPAIR INTERPRETATION

The repair ladder contains two qualitatively different interventions. Target-lowering interventions, such as q90 and q95 support calibration, change the effective prompt. They are deployment decisions: “this prompt is not supported, so lower it.” Likelihood and pilot calibration interventions keep the high target but change which candidate is selected. They are selection repairs: “this prompt is risky, so select a candidate whose behavior likelihood or pilot-predicted real utility is credible.”

This distinction matters for reviewer interpretation. It would be misleading to say q90 calibration “beats” raw high-target selection, because it is solving a different prompt. It is fair to say it reduces risk and keeps the policy inside the support envelope. It is fair to compare likelihood and pilot selectors directly against raw high-target selection because they operate on the same high-target candidate pool.

A.14 REPAIR LADDER ACROSS $N = 64, 128, 256$

A.15 CLAIM THRESHOLDS

The expanded claim JSON records the checks in Table 17. These thresholds are deliberately tied to the paper’s claims. If a future edit weakens one of the effects, the claim audit should fail rather than silently preserving old prose.

Table 15: **Expanded target-overshoot sweep, $N = 1$ and $N = 256$.**

Gap	N	target	proxy S	real R	unsafe rate
0.00	1	6.314	6.333	6.329	0.012
0.00	256	6.314	6.514	6.456	0.063
0.75	1	7.064	7.057	6.996	0.073
0.75	256	7.064	7.264	7.135	0.133
1.50	1	7.814	7.806	7.680	0.258
1.50	256	7.814	8.011	7.629	0.310
2.25	1	8.564	8.553	8.005	0.534
2.25	256	8.564	8.756	7.718	0.560
3.00	1	9.314	9.290	7.897	0.802
3.00	256	9.314	9.496	7.362	0.833
3.75	1	10.064	10.037	6.810	0.942
3.75	256	10.064	10.241	5.893	0.925
4.50	1	10.814	10.796	4.753	0.986
4.50	256	10.814	10.984	3.578	0.984

Table 16: **Repair ladder at high candidate counts.**

Selector	N	proxy S	real R	risk
Naive	64	9.464	7.410	0.0170
Naive	128	9.482	7.421	0.0171
Naive	256	9.501	7.303	0.0183
Behavior q05	64	9.193	8.505	0.0058
Behavior q05	128	9.196	8.501	0.0059
Behavior q05	256	9.195	8.519	0.0056
Pilot-calibrated	64	9.181	8.376	0.0068
Pilot-calibrated	128	9.177	8.435	0.0063
Pilot-calibrated	256	9.173	8.449	0.0060
Oracle real	64	9.226	8.501	0.0060
Oracle real	128	9.234	8.530	0.0058
Oracle real	256	9.237	8.542	0.0058

A.16 CARTPOLE-V1 BENCHMARK ARTIFACTS

The CartPole-v1 tier is regenerated by `experiments/run_cartpole_benchmark.py`. It writes trial rows, summary rows, claims, and a manifest under `results/cartpole_benchmark/`, plus Figure 10. Each of the 32 trials uses one shared initial state for the whole candidate pool so that selector differences are not explained by easier or harder starts.

The benchmark is intentionally not a D4RL-style leaderboard. Its purpose is adversarial diagnosis. The raw selector is allowed to see a proxy that is locally sensible but globally misspecified: it rewards short-horizon pole stability and late one-sided action pressure. The support-aware baselines then test whether the failure is due to missing support rather than an empty candidate pool. Since the behavior clone and oracle both reach the capped return while raw high- N selection collapses, the benchmark supports the same scoped mechanism claim as the synthetic microscope.

A.17 EXPANDED DESIGN MATRIX

The expanded suite was designed as a set of reviewer attacks rather than a single larger run. Each family asks whether a different explanation can defeat the claim. The candidate-count family attacks the possibility that the $N = 64$ result is a shallow artifact. The target-overshoot family attacks the support-boundary story. The repair family attacks whether the failure is recoverable from the same candidate pool. The pilot and noise families attack operational sensitivity.

Table 17: **Expanded claim checks.**

Check	observed	threshold	pass
Proxy return rises through $N = 256$	0.224	> 0.100	yes
Real utility drops through $N = 256$	-0.629	< -0.250	yes
Selected risk rises through $N = 256$	0.0071	> 0.005	yes
Pilot repair recovers raw tail	1.146	> 0.450	yes
Behavior filter recovers raw tail	1.216	> 0.450	yes
Oracle gap shows recoverable candidates	1.239	> 0.600	yes
High-gap target overshoot worsens at high N	-1.174	< -0.200	yes
Pilot size changes calibrated real utility	0.443	> 0.030	yes
Noise scale changes unsafe-action rate	0.113	> 0.020	yes

Table 18: **CartPole-v1 claim gates.** These checks are read directly by the repository audit.

Check	observed	threshold	pass
Raw proxy rises from $N = 1$ to $N = 64$	0.075	> 0.040	yes
Raw true return drops from $N = 1$ to $N = 64$	-263.2	< -120.0	yes
Support filter recovers over raw $N = 64$	454.1	> 150.0	yes
Conservative selector recovers over raw $N = 64$	438.7	> 150.0	yes
Behavior clone beats raw $N = 64$	454.1	> 150.0	yes
Oracle gap is visible	454.1	> 150.0	yes
Raw exploit-policy rate at $N = 64$	0.969	> 0.500	yes

A.18 STATISTICAL PRECISION OF THE HIGH-CANDIDATE TAIL

The expanded candidate-count rows use 42 independent replicates per N . Table 20 reports standard errors and approximate 95% half-widths for selected real utility. The $N = 1$ to $N = 256$ real-utility drop is 0.629, while the approximate half-widths are 0.103 and 0.182 respectively. The effect is therefore not a visual impression from a single curve. The uncertainty is largest at high N because top-score selection is deliberately hunting the noisy extreme tail.

The same table also shows why proxy-only reporting is misleading. Proxy return rises smoothly as N grows. Real utility has a downward trend with non-monotonic local variation, which is exactly what the theory predicts when the selected tail contains heterogeneous real utilities. Larger candidate pools increase pressure toward high proxy values; they do not guarantee monotone movement in real utility.

A.19 PER-REPLICATE FAILURE SLICE

The mean drop in Table 20 is backed by a visible bad tail. At $N = 256$, the selected-real distribution has a 10th percentile of 6.551, median 7.271, and 90th percentile 7.914. Selected risk has a 10th percentile of 0.0131, median 0.0185, and 90th percentile 0.0247. Table 21 lists the five lowest-real-utility selected trajectories. Each row still has high proxy return, so these are not failed prompt-satisfaction cases; they are high-proxy trajectories whose action risk and behavior likelihood reveal the hidden cost.

This failure slice is also a useful sanity check on the mechanism. The selector is not choosing obviously nonsensical proxy values. The selected proxy values are tightly concentrated near 9.5 and the maximum action values remain inside the clipping range. The problem is subtler: the selected sequences use enough near-boundary and beyond-boundary actions to incur real penalties that the proxy does not price correctly.

A.20 TARGET-GAP DERIVATIVE TABLE

Table 22 reports the $N = 256$ minus $N = 1$ change at each target gap. The first two rows are benign: high candidate count improves both proxy and real utility near the support boundary. The sign turns around once the target is outside support. At gap 4.5, proxy still improves by 0.188, while real utility drops by 1.174, risk rises by 0.0113, and behavior log-likelihood falls by 0.651.

Table 19: **Expanded experiment design matrix.** Every family is generated by the public expansion script and written to the same summary CSV.

Family	Grid	Main stress	Reviewer question
Candidate count	$N = 1$ to 256	fixed $q_{95} + 3$ target	Does the out-of-support failure survive beyond the original $N = 64$ range?
Target overshoot	gaps 0 to 4.5	$N = 1$ versus 256	Is the support boundary actually where prompt satisfaction and real utility separate?
Repair ladder	selectors $N = 64, 128, 256$	at same high target plus support-lowered prompts	Which repairs change the selected tail, and which simply refuse the unsafe prompt?
Pilot labels	4 to 64 labels	calibrated selector at $N = 256$	How much direct utility information is needed before calibration becomes stable?
Noise stress	scales 0.16 to 0.64	raw high-target selection	Does stochastic decoding expose more unsupported tail mass?

Table 20: **Precision of the expanded candidate-count stress.** Half-width is 1.96 times the standard error across replicates.

N	real mean	real SE	95% half-width	proxy mean	risk mean
1	7.888	0.052	0.103	9.283	0.0116
16	7.439	0.075	0.146	9.433	0.0166
64	7.353	0.086	0.168	9.469	0.0176
128	7.380	0.064	0.126	9.483	0.0175
256	7.258	0.093	0.182	9.507	0.0187

This table is the strongest compact argument against a duplicate “best-of- N wrapper” reading. The object being swept is not a generic sample count alone. It is a return-to-go prompt measured relative to offline return support. The central empirical claim is a boundary claim: candidate-count pressure is useful while the prompt remains behavior-supported, and it becomes dangerous when prompt satisfaction can be achieved through unsupported action tails.

A.21 SUPPORT METRICS ARE NOT REDUNDANT

The expanded trials record proxy return, real utility, risk, behavior log-likelihood, unsafe-action rate, mean action, and maximum action. These diagnostics are intentionally overlapping but not interchangeable. Table 23 reports per-trial correlations with selected real utility for several high- N slices. In the raw out-of-support tail, selected risk is almost perfectly anti-correlated with real utility, while behavior log-likelihood is strongly positively correlated. The unsafe-action rate is less reliable because many trajectories are already unsafe by a binary threshold; severity matters more than the count alone.

The repair rows explain why the paper reports multiple diagnostics. Once a behavior-likelihood filter has already truncated the candidate set, log-likelihood no longer explains much residual variation in real utility. Once a pilot model has learned from real labels, proxy return becomes more informative because the selector is no longer naively maximizing proxy alone. A reviewer should therefore read the diagnostics jointly: risk identifies the penalty channel, log-likelihood identifies support, and proxy identifies prompt satisfaction.

Table 21: **Worst selected trajectories under naive $N = 256$ out-of-support selection.** Rows are individual replicates from `expanded_trials.csv`.

Rep.	proxy	real	risk	logp	unsafe	mean act.	max act.
6	9.570	5.549	0.0334	-5.977	0.750	0.815	1.250
34	9.511	5.699	0.0318	-5.814	0.667	0.810	1.179
33	9.496	6.100	0.0283	-5.727	0.750	0.808	1.158
27	9.507	6.352	0.0263	-5.652	0.667	0.809	1.131
38	9.512	6.533	0.0248	-5.639	0.833	0.810	1.190

Table 22: **Change from $N = 1$ to $N = 256$ across target gaps.** Negative real deltas identify where high candidate count becomes harmful.

Gap above q_{95}	Δ proxy	Δ real	Δ risk	Δ logp
0.00	0.182	0.127	0.0005	-0.281
0.75	0.207	0.138	0.0006	-0.310
1.50	0.205	-0.051	0.0021	-0.392
2.25	0.203	-0.287	0.0041	-0.469
3.00	0.206	-0.535	0.0062	-0.535
3.75	0.204	-0.917	0.0093	-0.623
4.50	0.188	-1.174	0.0113	-0.651

A.22 REPAIR DECOMPOSITION AT HIGH CANDIDATE COUNT

Table 24 decomposes each $N = 256$ repair relative to naive high-target selection. Same-target selection repairs improve real utility while lowering risk and giving up a modest amount of proxy return. Target-lowering rows lower risk even more, but also solve an easier prompt and therefore are not claimed as same-objective improvements. This distinction is important because it prevents the paper from overstating the repair ladder.

The random row is a useful negative-control style observation. Random selection from the high-target candidate pool recovers 0.582 real-utility points relative to naive selection because it avoids the most extreme proxy tail. That does not make random selection a good deployment strategy. It demonstrates that the candidate pool contains better real-utility trajectories and that the failure is caused by the selection rule’s preference for proxy-extreme candidates.

A.23 PILOT-LABEL EFFICIENCY

The pilot-calibrated selector is deliberately presented as a limited-label diagnostic. Table 25 reports the real-utility mean, incremental improvement, and standard error as the number of real-utility pilot labels grows. The largest gain is from four to eight labels, and the result stabilizes by 32 labels. This supports a practical audit recommendation: before using large candidate counts on out-of-support prompts, collect enough direct utility labels to test whether the proxy tail is aligned.

The pilot result should not be read as a universal label-budget law. It is a measured property of this controlled environment and feature set. Its role is to show that direct real-utility feedback can identify the bad proxy tail with a small but nonzero budget, and to make the cost of that evidence explicit.

A.24 ROLLOUT-NOISE STRESS AS AN INFERENCE-TIME ATTACK

Table 26 reports how the naive $N = 256$ minus $N = 1$ effect changes with rollout noise. Larger noise increases the selected proxy gain, but it also increases the real-utility loss and risk rise. At noise scale 0.64, the selector obtains 0.256 extra proxy return while losing 1.010 real-utility points. This is an inference-time attack on the paper’s own claim: if the failure only appeared at a single decoding noise, it would be much easier to dismiss. Instead, higher stochasticity makes more unsupported tails available.

Table 23: **Per-trial diagnostic correlations with selected real utility.** Correlations are computed within each high- N slice.

Slice	proxy	risk	logp	unsafe rate
Naive candidate-count, $N = 256$	-0.129	-0.999	0.927	0.562
Target-overshoot, $N = 256$	-0.572	-0.881	0.684	-0.468
Repair ladder naive, $N = 256$	-0.111	-0.998	0.918	0.670
Behavior q05 repair, $N = 256$	0.102	-0.567	-0.009	0.536
Pilot-calibrated repair, $N = 256$	0.442	-0.865	-0.173	0.681

Table 24: **Repair deltas relative to naive $N = 256$ high-target selection.** Positive Δ real is better; negative Δ risk is safer.

Selector	Δ real	Δ risk	Δ proxy	Δ logp
Random	0.582	-0.0066	-0.214	0.567
Naive q90	-1.608	-0.0183	-3.800	5.575
Naive q95	-0.811	-0.0181	-2.982	4.868
Behavior q10	1.209	-0.0125	-0.294	0.893
Behavior q05	1.216	-0.0127	-0.306	0.918
Pilot calibrated	1.146	-0.0123	-0.328	0.943
Oracle real	1.239	-0.0125	-0.264	0.840

The unsafe-rate column is intentionally included even though it is not monotone. It reinforces the point from Table 23: binary threshold counts are a coarse support diagnostic. The risk magnitude and behavior likelihood carry the actual failure signal.

A.25 RAM-LIGHT REPRODUCIBILITY POLICY

The expanded suite is intentionally sequential. It does not keep all candidate trajectories from all families in memory at once. Each replicate samples a candidate pool, records only selected-candidate diagnostics and summary statistics, and writes compact CSV and JSON artifacts. Figures are generated from the summary table, not from a retained in-memory trajectory bank. This keeps the expansion compatible with ordinary laptop hardware while still increasing experimental scope.

This matters for the claims. A large but unreproducible run would not make the paper more submission-ready. The acceptance target is instead: enough replicates and stress axes to attack the mechanism, deterministic scripts that regenerate every table, and artifacts small enough for a reviewer or fresh agent to inspect. The repository therefore treats `expanded_summary.csv`, `expanded_trials.csv`, and `claims.json` as audit artifacts rather than temporary notebooks.

A.26 ADDITIONAL REVIEWER ATTACK CHECKLIST

Table 27 records the strongest attacks that remain after the expanded suite and CartPole-v1 benchmark tier, plus the exact evidence used to answer them. The point is not to claim that every attack is fully solved. The point is to ensure that every paper claim is no broader than the evidence that survives these attacks.

A.27 SELF-ATTACK LOG

The following attacks were used to harden the v4 manuscript:

1. **This is only a synthetic environment.** The paper now adds CartPole-v1 while still avoiding D4RL, MuJoCo, Atari, or deployment-scale claims.
2. **The finite-pool law is not novel enough.** Correct as a standalone theorem. The law is audit bookkeeping; the contribution is return-support diagnosis for DT prompts.

Table 25: **Pilot-label efficiency at $N = 256$.** Increment is relative to the previous label budget.

Pilot labels	real utility	increment	real SE
4	8.077	–	0.054
8	8.369	0.291	0.022
16	8.453	0.084	0.019
32	8.517	0.064	0.009
64	8.520	0.004	0.009

Table 26: **Noise stress deltas from $N = 1$ to $N = 256$.**

Noise	Δ proxy	Δ real	Δ risk	Δ unsafe
0.16	0.160	-0.420	0.0048	0.012
0.32	0.188	-0.608	0.0066	-0.010
0.48	0.219	-0.782	0.0083	-0.004
0.64	0.256	-1.010	0.0105	0.010

3. **The model is tiny.** Correct. It is a learned DT-style component sufficient for a controlled mechanism study, not a competitive Transformer.
4. **Increasing N helps in some settings.** Correct and reported. In-support and edge-support rows help; this is why the paper uses a Tail Phase Diagram rather than a universal harm claim.
5. **Support calibration changes the prompt.** Correct. The paper treats it as a deployment gate, not as a same-target optimizer.
6. **Pilot calibration needs real labels.** Correct. The label budget ablation makes this cost explicit.
7. **The target overshoot result is non-monotonic.** Correct. Moderate overshoot can help; far-out overshoot collapses. This nuance is central.
8. **Unsafe-action rate is high even at $N = 1$.** Correct. The v4 claim uses selected risk, behavior likelihood, CartPole action imbalance, and real utility rather than only unsafe rate.
9. **Noise stress could be a decoding artifact.** Correct. That is why it is reported as an inference-time stress, not as a property of all DT decoders.
10. **The paper could look like other best-of- N papers.** The manuscript avoids this by foregrounding return prompts, offline support quantiles, behavior likelihood, pilot real labels, and support-target gates.

A.28 OPERATIONAL AUDIT PROTOCOL

For a new return-conditioned policy, the audit should proceed as follows:

1. Estimate the empirical return support of the offline data, at least q_{50} , q_{80} , q_{90} , q_{95} , q_{99} , and maximum observed proxy return.
2. For each target return, record target minus support boundary.
3. Sample candidate action sequences for several N values, including a small N baseline and a high- N stress point.
4. Log selected proxy return, real utility, action risk, behavior likelihood, and prompt-satisfaction gap.
5. Include supported targets where candidate count is allowed to help.
6. Include out-of-support targets where prompt satisfaction may become misleading.
7. Compare raw proxy selection against behavior-likelihood filters, target-lowering gates, pilot real-utility calibration, and oracle real utility.
8. Report where support calibration changes the prompt rather than pretending it solves the same objective.

Table 27: Reviewer attack checklist after the expanded suite and CartPole-v1 tier.

Attack	Evidence in this artifact	Remaining limitation
The model is too small.	The paper calls it DT-style and focuses on the return-conditioning interface; claims are mechanism-level.	Benchmark-scale Transformers remain future work.
The result is synthetic.	Synthetic setup exposes proxy reward, real utility, support boundary, and action risk exactly; CartPole-v1 repeats the failure under standard control dynamics.	D4RL, MuJoCo, Atari, and deployment-scale claims are not made.
The theorem is generic.	The empirical object is target-return overshoot relative to offline support, not only top-score accounting.	The accounting identity is supporting machinery, not the sole novelty.
The original $N = 64$ result may be accidental.	Candidate-count stress extends to $N = 256$ with proxy up, real down, risk up, and logp down.	Larger N values could be explored but are not needed for the stated failure.
Candidate selection can help.	In-support and edge-support targets improve under high candidate counts.	The paper makes a conditional boundary claim, not a universal harm claim.
Repairs are cherry-picked.	The ladder includes random, target-lowered, likelihood-filtered, pilot-calibrated, and oracle selectors.	Richer support estimators may improve over these simple diagnostics.
Pilot calibration is expensive.	Label-size ablation reports gains from 4 through 64 labels and shows diminishing returns.	Real deployments may have noisier labels or delayed outcomes.
Unsafe rate is confusing.	Correlation analysis shows why risk magnitude and behavior likelihood are more informative.	Better multidimensional support diagnostics are future work.
The manuscript duplicates other candidate-count papers.	The evidence centers return-to-go prompts, offline return quantiles, target overshoot, support gates, pilot real utility, and CartPole-v1 support failures.	Large-scale DT experiments would extend the scope, not define the current claim.

A.29 EXTERNAL VALIDITY PLAN

A stronger future paper should move along four axes. First, replace the one-dimensional synthetic action process with standard offline-control environments while preserving the return-support diagnostics. Second, evaluate larger sequence models and modern DT variants. Third, test whether support-gap and behavior-likelihood diagnostics predict real execution failures under richer dynamics. Fourth, evaluate how many pilot real-utility labels are required before calibration becomes reliable under distribution shift.

The present artifact intentionally covers only the first controlled mechanism layer. Its value is that every number is auditable and every claim is narrow.

A.30 FINAL RELEASE CHECKLIST

Before release, the repository should satisfy:

1. the original full suite regenerates the baseline figures and tables;
2. the expanded suite regenerates `results/expansion/claims.json` with `all_passed=true`;
3. the CartPole-v1 benchmark regenerates `results/cartpole_benchmark/claims.json` with `all_passed=true`;
4. the final PDF is built at `paper/final/best_of_n_decision_transformer-v4.pdf`;
5. the final PDF has at least 25 pages;

-
6. the LaTeX log has no undefined references, citation warnings, overfull boxes, fatal errors, or emergency stops;
 7. the claim audit returns `submission-ready v4`;
 8. the Desktop PDF hash matches the repository final PDF hash;
 9. the Desktop source map points this paper to the source folder and GitHub repository.

A.31 FRESH-AGENT REPRODUCTION MANIFEST

The final artifact is intended to be inspectable by a fresh agent without private context. A fresh run should begin from the repository root, install the package in the usual local environment if needed, and execute the verification commands in this order:

1. `python -m compileall src tests experiments -q`;
2. `pytest -q`;
3. `bash scripts/run_all.sh`;
4. `python -m experiments.run_cartpole_benchmark`;
5. `bash scripts/run_claim_audit.sh`.

The expected terminal-level outcome is that the test suite passes, the full run and CartPole benchmark regenerate passing claim JSON files, and the claim audit reports `submission-ready v4`. The final PDF should then be rebuilt with the paper build script and checked for page count and LaTeX log cleanliness.

The minimum artifact set for a reviewer or fresh agent is:

1. the final PDF in `paper/final`;
2. the original summary, sweep, repair, anti-aligned, and accounting CSV files in `results`;
3. the expanded summary, trial, claims, and manifest files in `results/expansion`;
4. the CartPole benchmark artifacts in `results/cartpole.benchmark`;
5. the ten paper figures in `figures`;
6. the claim-status JSON and markdown files;
7. the source files for the model, data generator, repair selectors, metrics, and audit.

If any one of these artifacts is missing, the manuscript can still be read, but it is no longer fully submission-auditable. The paper's strongest form is the coupled package: executable experiments, generated evidence, a final manuscript, and a claim audit that can fail unsupported edits.